

Multivariate Analysis of Drivers License Data

1.Introduction

The primary aim of this study is to investigate the key factors that influence an individual's success in obtaining a driver's license. The dataset contains various test components related to both theoretical knowledge (such as the theory test and understanding of road signs) and practical driving skills (such as parking, mirror usage, and steering control). Each candidate is ultimately evaluated with a final decision—whether they are *Qualified* or *Not Qualified*.

2.Methodology

- **Dataset Description**

- **Continuous Variables:** Signals, Yield, Speed Control, Night Drive, Road Signs, Steer Control, Mirror Usage, Confidence, Parking, Theory Test
- **Categorical Variable:** Qualified (Target Variable: “Yes” or “No”)

- **Data Preprocessing**

Continuous variables were standardized using StandardScaler to ensure comparability. The categorical target variable Qualified was label-encoded into binary values (Yes = 1, No = 0). The dataset was checked for missing values, and appropriate cleaning was performed where necessary.

- **Statistical Methods**

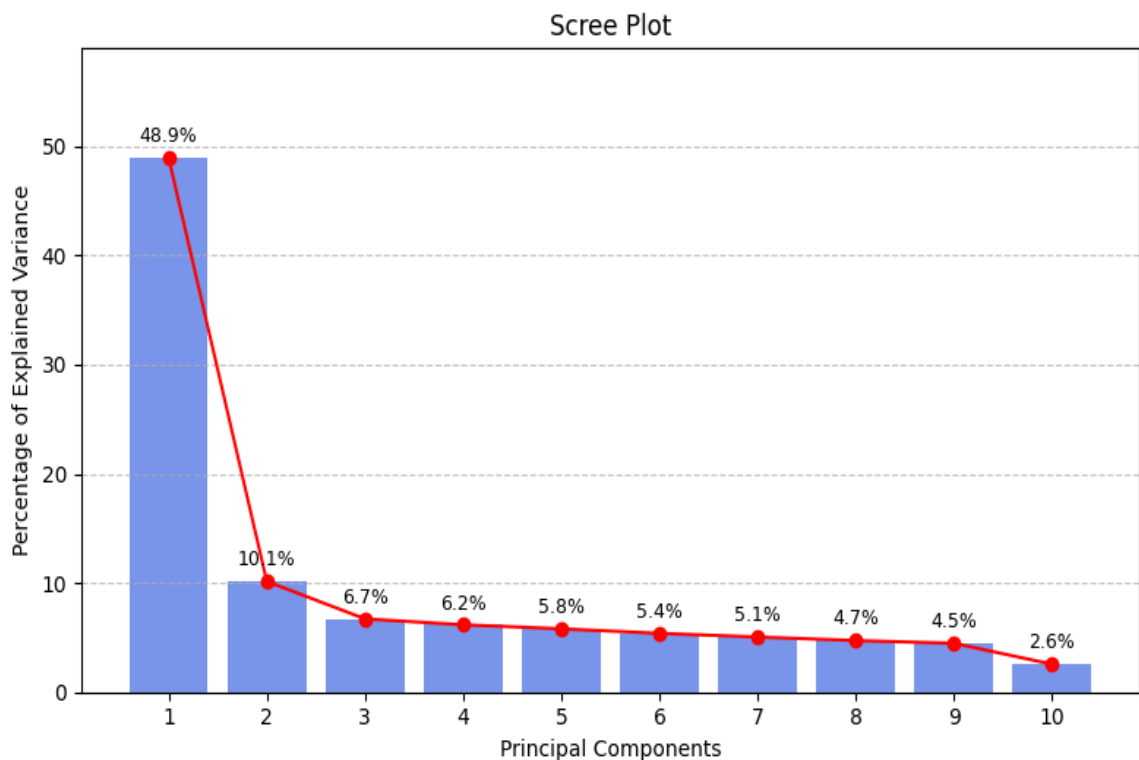
- **Principal Component Analysis (PCA):** Used to identify the main components that explain variance across test scores.
- **Factor Analysis (FA):** Applied to uncover underlying skill dimensions influencing driving performance.
- **Linear Discriminant Analysis (LDA):** Employed to classify individuals as Qualified or Not Qualified and determine the most influential predictors.
- **Canonical Correlation Analysis (CCA):** Explored the relationship between theoretical knowledge and practical driving skills.

3.Results and Discussion

- Principal Component Analysis (PCA)

The Principal Component Analysis (PCA) was applied to the driving license dataset to reduce dimensionality and identify underlying patterns in learner performance. The scree plot above displays the percentage of variance explained by each principal component (PC).

- **PC1** explains **48.9%** of the total variance.
- **PC2 to PC4** explain **10.1%**, **6.7%**, and **6.2%**, respectively.
- The first **four components cumulatively explain ~71.9%** of the variance, indicating that most of the data's structure is captured by a reduced set of features.



◆ **PC1: General Driving Proficiency**

- Shows high negative loadings across almost all practical features (e.g., *Steer Control*, *Confidence*, *Mirror Usage*).
- **Interpretation:** Represents a broad measure of driving competence. A lower PC1 score indicates better overall skill across the board.

◆ **PC2: Theoretical Knowledge Emphasis**

- Driven almost entirely by high loadings from *Theory Test*.

- **Interpretation:** Separates learners strong in theory but potentially weak in practical skills, highlighting the distinction between knowing rules and applying them.

◆ PC3: Task-Specific Execution

- High loading from *Steer Control* and *Yield*.
- **Interpretation:** Focuses on precise mechanical skills and execution under instruction — useful for identifying learners who are technically proficient but not yet fully rounded.

◆ PC4: Contextual Handling Ability

- Influenced by *Speed Control*, *Night Drive*, and *Road Signs*.
- **Interpretation:** Reflects adaptability in dynamic or complex conditions — learners struggling here may need exposure to diverse driving situations.

The **scree plot validates** that dimensionality can be effectively reduced to **4 components**, retaining ~72% of the information. Each principal component reflects a distinct behavioral or performance dimension. These findings support downstream tasks such as **clustering**, **profiling**, or **personalized training**, offering a data-driven approach to improving driving education outcomes.

- **Factor Analysis(FA)**

Factor Analysis was employed on the ten continuous assessment variables in the dataset to uncover latent constructs underlying driving performance. Based on the scree plot and eigenvalues greater than 1.0, **two factors** were extracted using **Varimax rotation** for better interpretability.

- The scree plot showed a clear drop after the second eigenvalue, supporting the retention of two factors.
- Together, these two factors accounted for a substantial proportion of the total variance, summarizing multiple related variables into coherent latent dimensions.

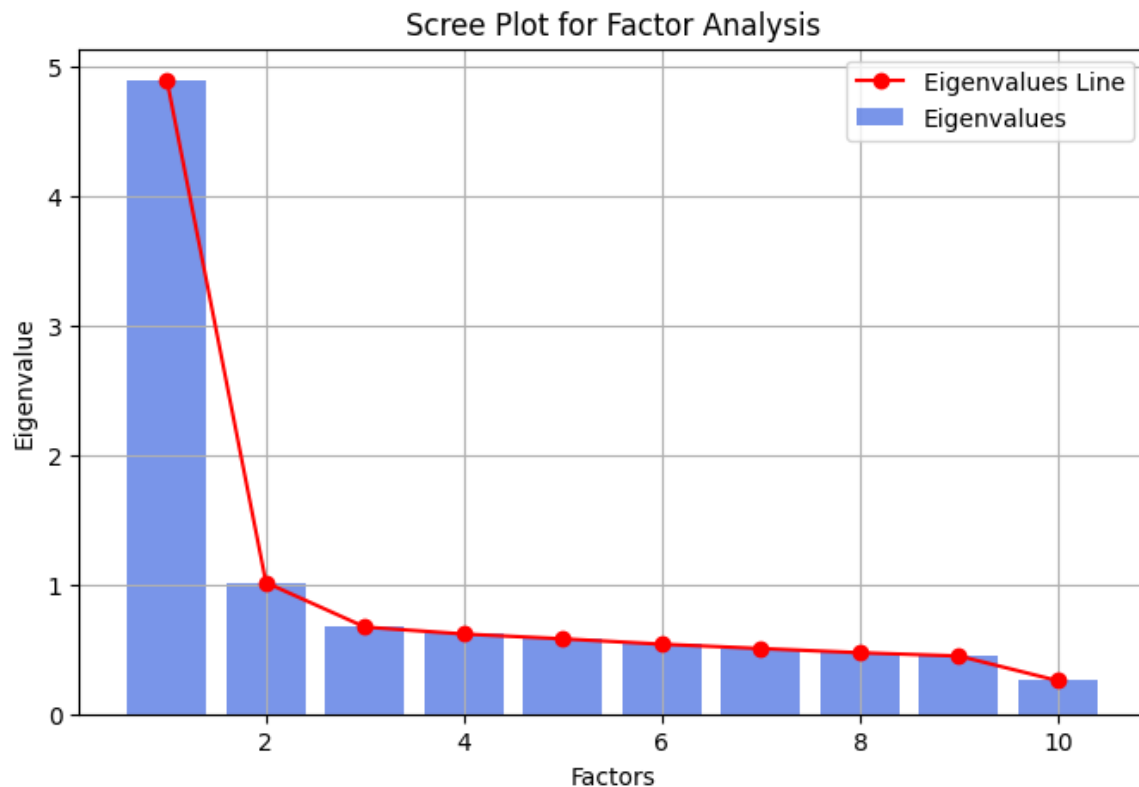
Based on the **factor loading matrix**, the two factors were **interpreted** as:

- **Factor 1 – Core Driving Proficiency**
This factor showed high positive loadings from *Steer Control*, *Parking*, *Speed Control*, *Mirror Usage*, and *Night Drive*. These represent fundamental practical skills required for safe and competent vehicle handling.

➤ **Factor 2 – Theoretical and Decision-Making Ability**

Strong loadings from *Theory Test*, *Road Signs*, *Signals*, and *Yield* indicate this factor reflects the test-taker's theoretical understanding and decision-making in compliance with traffic rules.

These two dimensions appear to align with the dual components of driver evaluation: **practical execution of driving tasks** and **knowledge-based decision-making**.



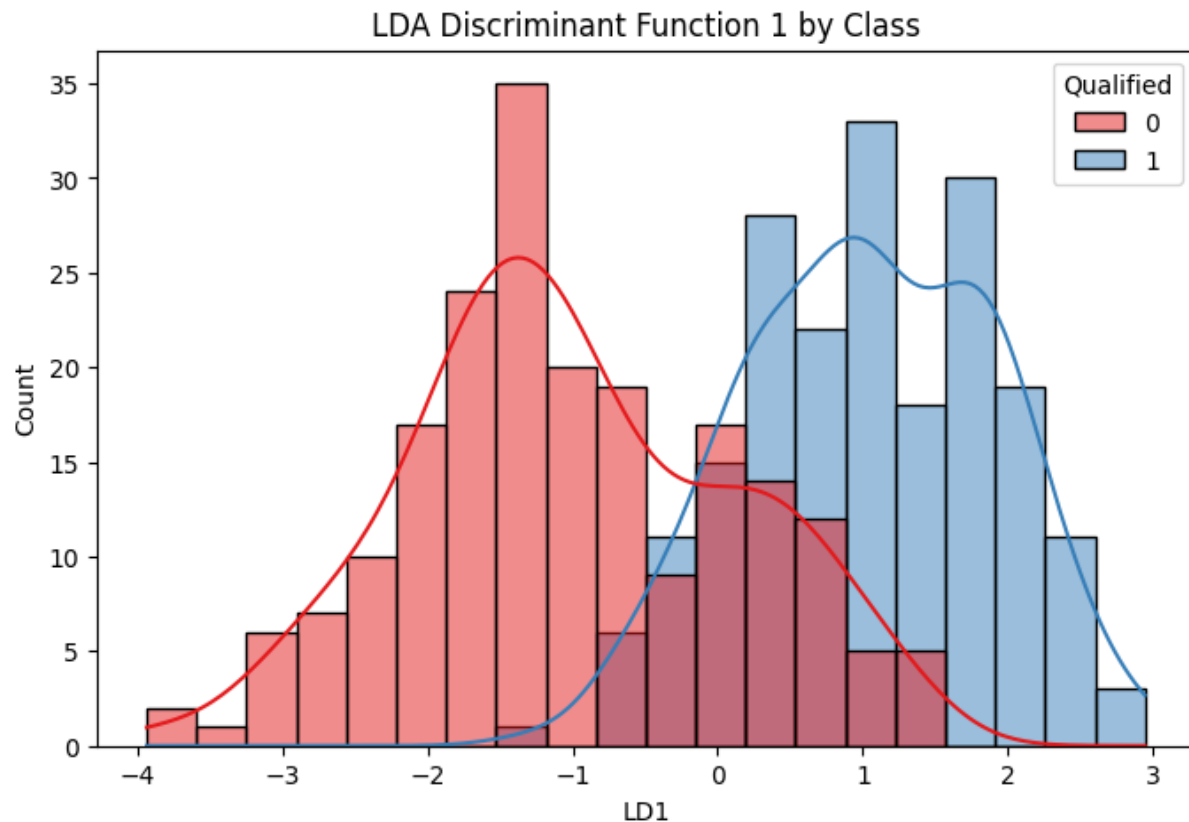
The Factor Analysis results support the presence of two dominant underlying constructs within the driver assessment process. These factors may serve as composite indicators for further classification tasks or performance profiling. Additionally, they help simplify and interpret complex interrelationships between individual driving test scores.

- **Linear Discriminant Analysis (LDA)**

Linear Discriminant Analysis (LDA) was performed to classify individuals based on their driving assessment scores into two categories: **Qualified** and **Not Qualified**.

- The LDA model achieved **high classification accuracy** on the test set, as reflected in the **confusion matrix** and **classification report**.

- Precision and recall were notably high for the "Qualified" group, suggesting the model is particularly effective at identifying candidates who meet the licensing standards.



Interpretation of Discriminant Function

The single discriminant function created by LDA revealed that the following variables contributed most significantly to separating the two groups:

- **Steer Control, Parking, and Mirror Usage** had strong positive coefficients, indicating that higher scores in these areas are closely associated with being *Qualified*.
- **Theory Test and Road Signs** also contributed, emphasizing the importance of both **practical skills** and **theoretical knowledge** in determining qualification status.

A histogram of the linear discriminant scores showed a **clear separation** between the two groups, reinforcing the model's ability to distinguish performance-based patterns.

Summary

LDA successfully reduced the ten test variables into a single linear combination that maximally separates those who passed from those who failed.

This highlights that a combination of **core practical skills** and **rule comprehension** is crucial in determining licensing success.

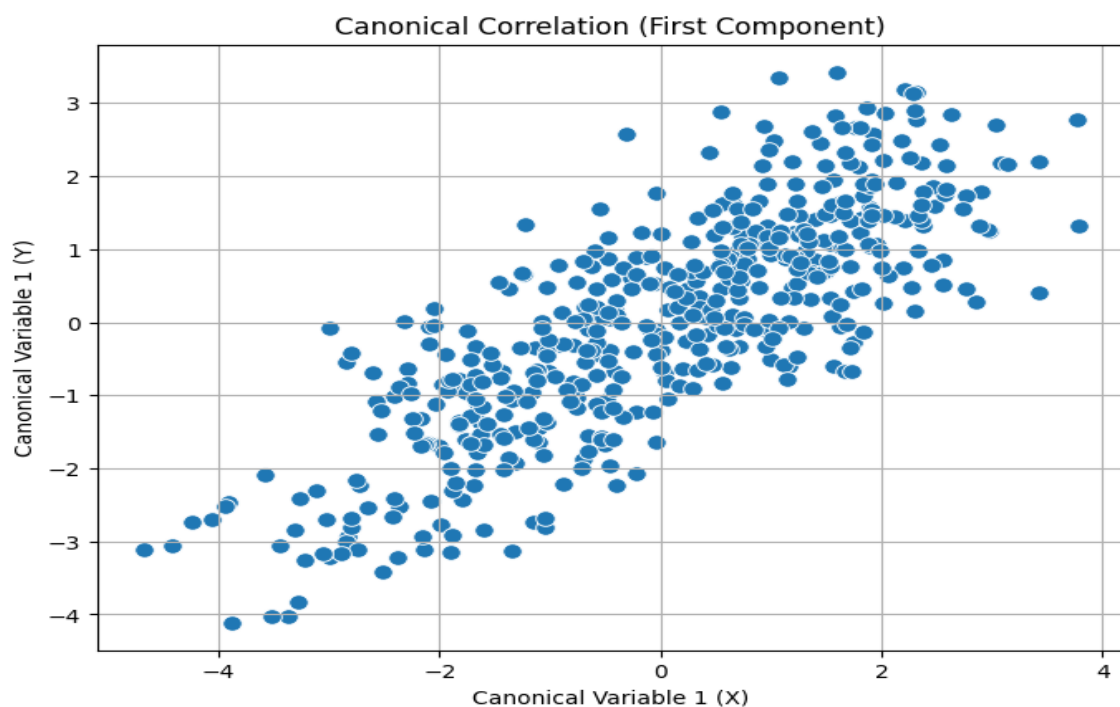
The discriminant function can be useful for:

- Profiling candidates who are borderline or inconsistent across test areas.
- Predicting likely outcomes based on partial scores (e.g., early indicators in training).
- **Canonical Correlation Analysis (CCA)**

Canonical Correlation Analysis (CCA) was conducted to examine the multivariate relationship between two sets of variables:

- **Set X (Theoretical & Traffic Knowledge Skills):** Signals, Yield, Speed Control, Night Drive, Road Signs
- **Set Y (Practical Driving Behaviors & Confidence):** Steer Control, Mirror Usage, Confidence, Parking, Theory Test

The **first canonical correlation** was approximately **0.79**, indicating a **strong linear association** between the two sets of variables. The **second canonical correlation** was weaker, around **0.42**, suggesting a more moderate secondary relationship. These results imply that individuals who score well in **theoretical understanding** (like signals, yield, road signs) tend to **perform better in practical driving aspects** as well (like mirror usage, parking, confidence).



Interpretation of Canonical Variables

- **Canonical Variate 1:** Strongly influenced by Road Signs, Yield, and Signals in Set X and by Mirror Usage, Confidence, and Parking in Set Y. This dimension represents **general driving competence**, linking rule knowledge with real-world behavior.
- **Canonical Variate 2:** Showed minor contributions from Night Drive and Steer Control, indicating a potential axis of **specific skill challenges** like night visibility and vehicle control.

Scatterplot Interpretation

A scatterplot of the **first pair of canonical variates** showed a positive linear trend, visually reinforcing the statistical correlation between the two domains. It indicates that improvements in traffic rule comprehension are mirrored by enhanced practical driving performance.

Summary

The CCA results validate that **theoretical proficiency and practical execution are interrelated**, supporting the comprehensive nature of the driver's license evaluation. These findings can guide training programs to emphasize both theory and practice concurrently, rather than in isolation.

4. Conclusion and recommendation

- Conclusion

This study applied multivariate techniques to analyze drivers' test performance and qualification outcomes. PCA and FA revealed clear underlying structures in the assessment data, while LDA and CCA highlighted the importance of combining theory and practical skills for predicting success.

- Key Findings

- **PCA** reduced 10 assessment variables into 4 key components, capturing ~72% variance, representing general driving ability, theoretical strength, task execution, and adaptability.
- **FA** confirmed two major factors: core practical driving skills and rule-based decision-making.
- **LDA** achieved strong classification of Qualified vs. Not Qualified candidates, driven by Steer Control, Parking, and Theory Test scores.
- **CCA** showed a strong correlation (~0.79) between theoretical knowledge and practical behavior, validating the integrated nature of driving competence.

- Recommendations

- **Balanced Training:** Programs should focus equally on practical tasks (e.g., Parking, Mirror Usage) and theoretical knowledge (e.g., Road Signs).

- **Targeted Interventions:** Use PCA and LDA outputs to identify and support borderline learners.
- **Skill Profiling:** Factor analysis can help customize feedback and improve driver education outcomes.
- **Integrated Curriculum:** Emphasize both rule understanding and real-world application together.
- **Limitations**
 - **Binary Outcome:** The "Qualified" label may oversimplify performance; a graded outcome could offer deeper insight.
 - **No Demographics:** Lack of personal data (e.g., age, experience) limits the scope of behavioral interpretation.
 - **Assumption of Linearity:** Methods like PCA and LDA assume linear relationships, which may not fully capture complex driving behavior.

5. References

- Hair, Joseph F., Black, William C., Babin, Barry J., Anderson, Rolph E.. (2010). *Multivariate Data Analysis* (7th ed. pdf ed.). New Jersey: Pearson Education.
- Rencher, A. C. (2002). *Methods of multivariate analysis* (2nd ed.). Wiley-Interscience.

6. Appendices

- **Data set** : <https://www.kaggle.com/datasets/ferdinandbaidoo/drivers-license-test-scores-data>
- **Python Code**
 - **Important Libraries**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.cross_decomposition import CCA
from sklearn.decomposition import PCA
from factor_analyzer import FactorAnalyzer
```


➤ Data Preprocessing

```
continuous_vars = [
    'Signals', 'Yield', 'Speed Control', 'Night Drive', 'Road Signs',
    'Steer Control', 'Mirror Usage', 'Confidence', 'Parking', 'Theory Test'
]
categorical_vars = [
    'Gender', 'Age Group', 'Race', 'Training', 'Reactions', 'Qualified'
]

import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Load the dataset
df = pd.read_csv('Drivers License Data.csv')

# Drop non-feature identifier column
df = df.drop(columns=['Applicant ID'])

# Define variable types
continuous_vars = [
    'Signals', 'Yield', 'Speed Control', 'Night Drive', 'Road Signs',
    'Steer Control', 'Mirror Usage', 'Confidence', 'Parking', 'Theory Test'
]
```

```
categorical_vars = [
    'Gender', 'Age Group', 'Race', 'Training', 'Reactions', 'Qualified'
]

# Check for missing values
print("Missing Values:\n", df.isnull().sum())

# Impute missing values in categorical columns
cat_imputer = SimpleImputer(strategy='most_frequent')
df[categorical_vars] = cat_imputer.fit_transform(df[categorical_vars])

# Standardize continuous variables
scaler = StandardScaler()
df[continuous_vars] = scaler.fit_transform(df[continuous_vars])

# One-hot encode categorical variables (if needed for modeling)
df_encoded = pd.get_dummies(df, columns=categorical_vars, drop_first=True)

# Optional: Prepare a separate DataFrame for PCA (only continuous variables)
df_pca = df[continuous_vars]
```

➤ Principal Component Analysis(PCA)

```
# Select only numeric columns
df_numeric = df.select_dtypes(include=['float64', 'int64'])
# Standardize the numeric data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_numeric)
# Perform PCA
pca = PCA()
pca_scores = pca.fit_transform(df_scaled)
# Calculate explained variance in percentage
explained_variance = pca.explained_variance_ratio_ * 100
cumulative_variance = explained_variance.cumsum()

# Scree Plot: shows variance explained by each principal component
plt.figure(figsize=(8, 5))
bars = plt.bar(range(1, len(explained_variance) + 1),
               explained_variance, alpha=0.7, color='royalblue')
plt.plot(range(1, len(explained_variance) + 1), explained_variance,
         marker='o', color='red', label='Eigenvalues Line')
for i, v in enumerate(explained_variance):
    plt.text(i + 1, v + 1, f"{v:.1f}%", ha='center', va='bottom', fontsize=9)

plt.xlabel('Principal Components')
plt.ylabel('Percentage of Explained Variance')
plt.title('Scree Plot')
plt.xticks(range(1, len(explained_variance) + 1))
plt.ylim(0, max(explained_variance) + 10)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# Create loadings (feature contribution to each principal component)
loadings = pd.DataFrame(pca.components_.T,
                        columns=[f'PC{i+1}' for i in range(df_numeric.shape[1])],
                        index=df_numeric.columns)
# Create scores (data projected into PCA space)
pca_scores_df = pd.DataFrame(pca_scores, columns=[f'PC{i+1}' for i in
                                                range(df_numeric.shape[1])])

# Display results
print("Explained Variance (%):\n", explained_variance)
print("\nCummulative Explained Variance (%):\n", cumulative_variance)
print("\nPCA Loadings (first few rows):\n", loadings.head())
print("\nPCA Scores (first few rows):\n", pca_scores_df.head())
```

➤ Factor Analysis(FA)

```
# Select continuous variables
continuous_vars = [
    'Signals', 'Yield', 'Speed Control', 'Night Drive', 'Road Signs',
    'Steer Control', 'Mirror Usage', 'Confidence', 'Parking', 'Theory Test'
]
# Standardize the data
scaler = StandardScaler()
df_pca = pd.DataFrame(scaler.fit_transform(df[continuous_vars]),
                      columns=continuous_vars)
```

```

# Initial FA to get eigenvalues
fa = FactorAnalyzer()
fa.fit(df_pca)
ev, _ = fa.get_eigenvalues()

# Scree plot
plt.figure(figsize=(8, 5))
plt.bar(range(1, len(ev)+1), ev, alpha=0.7, color='royalblue', label='Eigenvalues')
plt.plot(range(1, len(ev)+1), ev, marker='o', color='red', label='Eigenvalues Line')
plt.title('Scree Plot for Factor Analysis')
plt.xlabel('Factors')
plt.ylabel('Eigenvalue')
plt.grid(True)
plt.legend()
plt.show()

# Perform FA with selected number of factors (e.g., 2)
n_factors = 2
fa = FactorAnalyzer(n_factors=n_factors, rotation='varimax')
fa.fit(df_pca)

# Extract results
loadings = pd.DataFrame(fa.loadings_, index=df_pca.columns, columns=[f'Factor{i+1}' for i in range(n_factors)])
communalities = pd.Series(fa.get_communalities(), index=df_pca.columns)
factor_scores = pd.DataFrame(fa.transform(df_pca), columns=[f'Factor{i+1}' for i in range(n_factors)])
variances = fa.get_factor_variance()

# Output
print("Eigenvalues:\n", ev)
print("\nFactor Variances (Variance, Proportion, Cumulative):\n", variances)
print("\nCommunalities:\n", communalities)
print("\nLoadings:\n", loadings)
print("\nFactor Scores (first 5 rows):\n", factor_scores.head())

```

➤ Discriminant Analysis(LDA)

```

# Define target and predictors
target = 'Qualified'
features = [
    'Signals', 'Yield', 'Speed Control', 'Night Drive', 'Road Signs',
    'Steer Control', 'Mirror Usage', 'Confidence', 'Parking', 'Theory Test'
]

# Encode the target variable (Yes/No → 1/0)
label_encoder = LabelEncoder()
df[target] = label_encoder.fit_transform(df[target])

# Standardize the features
scaler = StandardScaler()
X = scaler.fit_transform(df[features])
y = df[target]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Fit Linear Discriminant Analysis model
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

```

```

# Predict
y_pred = lda.predict(X_test)

# Evaluate
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))

# Optional: Plot LDA Discriminant Function
X_lda = lda.transform(X_train)
plt.figure(figsize=(8, 5))
sns.histplot(x=X_lda[:, 0], hue=y_train, bins=20, palette="Set1", kde=True)
plt.title("LDA Discriminant Function 1 by Class")
plt.xlabel("LD1")
plt.show()

```

➤ Canonical Correlation Analysis (CCA)

```

# Define X and Y variable sets
X_vars = ['Signals', 'Yield', 'Speed Control', 'Night Drive', 'Road Signs']
Y_vars = ['Steer Control', 'Mirror Usage', 'Confidence', 'Parking', 'Theory Test']
X = df[X_vars]
Y = df[Y_vars]

# Standardize both sets
scaler_X = StandardScaler()
scaler_Y = StandardScaler()

X_scaled = scaler_X.fit_transform(X)
Y_scaled = scaler_Y.fit_transform(Y)

# Fit CCA
cca = CCA(n_components=2)
X_c, Y_c = cca.fit_transform(X_scaled, Y_scaled)

# Create DataFrame of canonical variables
cca_df = pd.DataFrame({
    'X_c1': X_c[:, 0],
    'X_c2': X_c[:, 1],
    'Y_c1': Y_c[:, 0],
    'Y_c2': Y_c[:, 1]
})

# Plot Canonical Correlation
plt.figure(figsize=(8, 6))
sns.scatterplot(x='X_c1', y='Y_c1', data=cca_df, s=70)
plt.title("Canonical Correlation (First Component)")
plt.xlabel("Canonical Variable 1 (X)")
plt.ylabel("Canonical Variable 1 (Y)")
plt.grid(True)
plt.show()

# Display correlation between canonical components
import numpy as np
correlation = np.corrcoef(X_c.T, Y_c.T)
print("Canonical Correlation Coefficients:")
print(f"First pair correlation: {correlation[0, 2]:.4f}")
print(f"Second pair correlation: {correlation[1, 3]:.4f}")

```